deterministic, as they can be added to any of the clusters from which they are reachable, and the one cluster to which they are eventually added depends on the order used to process dataset's points.

There is one last question we need to ask: how many times do we need to iterate DBSCAN's main loop? That will be exactly once per point. This is completely different from k-means, where we had many iterations of a few steps on the whole dataset: while k-means is a search heuristic adjusting some parameters to move to a local minimum[16], DBSCAN is a 1-pass deterministic algorithm computing the best partitioning (and at the same time identifying outliers) based on the points' density in different areas.

12.3.3 And Finally, an Implementation

Once we have outlined how the algorithm works at a high level, we are ready to write an implementation of DBSCAN, shown in listing 12.6.

If you recall, we have mentioned in chapter 11 that data structures like k-d trees and Ss-trees are often used in clustering: as for k-means, also for DBSCAN a multidimensional indexing structure is used to speed up range queries – you might have figured it out at the definition of *core points*, considering that to decide if a point p is a core point we need to check how many dataset points there are in its neighborhood, within a certain radius from p.

And while for k-means we need to perform nearest neighbors queries, for DBSCAN we will instead run range queries,